

系统开发工具基础

第四周实验报告

作者：魏尚儒

学号：25020007136

链接：

<https://github.com/1583e4d-Rn/2026summer-reports>

2026 年 9 月 14 日

目录

1	实验内容	2
1.1	建立可执行的本地质量门禁	2
1.2	让 Make 只重建真正受影响的产物	6
1.3	把本地 API 数据转换为可读报告	8
1.4	修复并交付一个陌生的小型工具仓库	11
2	课后练习	14
2.1	代码质量	14
2.1.1	在 pyproject.toml 中配置 ruff 规则	14
2.1.2	用 pytest 参数化测试减少重复	14
2.1.3	用 pytest.raises 断言异常与退出码	15
2.2	元编程：构建系统	15
2.2.1	GNU Make 的自动变量	15
2.2.2	用.PHONY 声明伪目标	15
2.3	大杂烩：API、jq 与 Markdown	16
2.3.1	curl 的常用选项	16
2.3.2	用 jq 筛选与排序 JSON	16
2.3.3	用 jq -r 拼接 Markdown 表格	16
2.4	Python 与 CLI 约定	16
2.4.1	用 argparse 定义必选参数	16
2.4.2	计算文件的 SHA-256 校验和	17
3	感悟	17

1 实验内容

1.1 建立可执行的本地质量门禁

本题要求复用第 10 题的 greetlab 包，建立一个轻量本地质量检查，详细如下：

- 复制 q10 为 q13；在 pyproject.toml 中加入 ruff 和 pytest 的最小配置。
- 补充一个正常姓名测试和一个空白姓名测试，每个测试包含有效断言。
- 运行 ruff format、ruff check 和 pytest，修复全部问题，不得全局忽略规则。
- 编写 check.sh，依次执行格式检查、静态检查和 pytest；运行并保证返回 0。

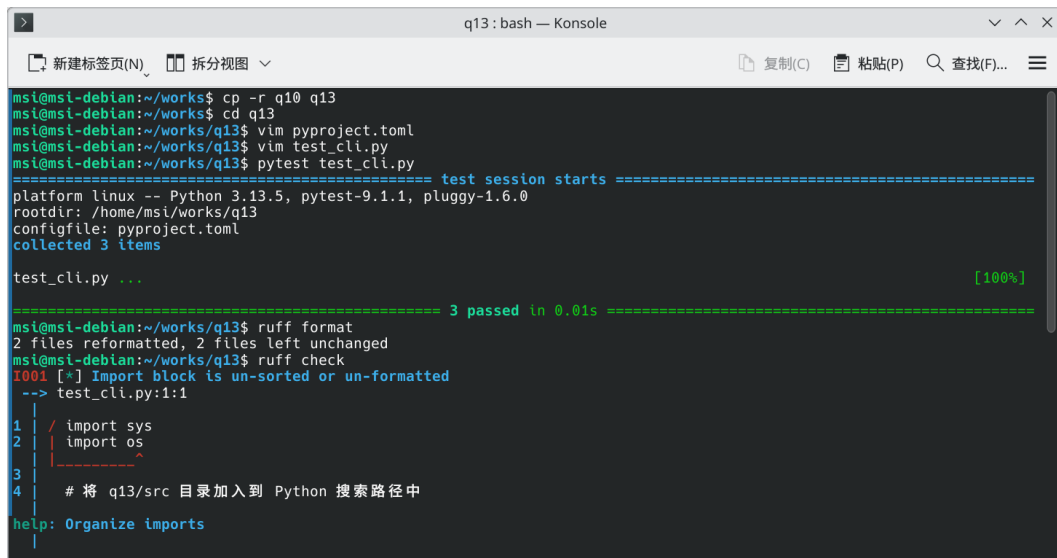
配置 ruff 与 pytest

在 pyproject.toml 中加入 ruff 的规则配置与 pytest 的测试路径配置：

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-25020007136"
version = "0.1.0"
[project.scripts]
sdt-greet = "greetlab.cli:main"
[tool.ruff]
line-length = 88
target-version = "py313"
[tool.ruff.lint]
select = ["E", "F", "I", "W"]
[tool.pytest.ini_options]
```

```
testpaths = ["."]
python_files = "test_*.py"
```



```
q13: bash — Konsole
新建标签页(N) 拆分视图
msi@msi-debian:~/works$ cp -r q10 q13
msi@msi-debian:~/works$ cd q13
msi@msi-debian:~/works/q13$ vim pyproject.toml
msi@msi-debian:~/works/q13$ vim test_cli.py
msi@msi-debian:~/works/q13$ pytest test_cli.py
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/msi/works/q13
configfile: pyproject.toml
collected 3 items

test_cli.py ... [100%]
===== 3 passed in 0.01s =====
msi@msi-debian:~/works/q13$ ruff format
2 files reformatted, 2 files left unchanged
msi@msi-debian:~/works/q13$ ruff check
I001 [*] Import block is un-sorted or un-formatted
--> test_cli.py:1:1
1 | / import sys
2 | | import os
3 | |
4 | # 将 q13/src 目录加入到 Python 搜索路径中
help: Organize imports
```

补充测试

补充正常姓名与空白姓名两个测试，每个测试都包含有效断言：

```
import os
import sys

# 将 q13/src 目录加入到 Python 搜索路径中
sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "src")))

import pytest

from greetlab.cli import main
```

```
def test_blank_name_exits_with_2(monkeypatch):
    # 模拟命令行输入
    monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", ""])
    with pytest.raises(SystemExit) as exc_info:
        main()
    # 验证退出码为 2
    assert exc_info.value.code == 2

def test_normal_name(monkeypatch, capsys):
    """ 测试正常姓名，应输出 Hello 问候语并正常退出。 """
    monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "Alice"])
    main()

    captured = capsys.readouterr()
    assert "Hello, Alice!" in captured.out

def test_blank_name(monkeypatch, capsys):
    """ 测试纯空白姓名，应以 SystemExit(2) 结束。 """
    monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", ""])
    with pytest.raises(SystemExit) as exc_info:
        main()

    assert exc_info.value.code == 2

    captured = capsys.readouterr()
    assert "error" in captured.err.lower()
```

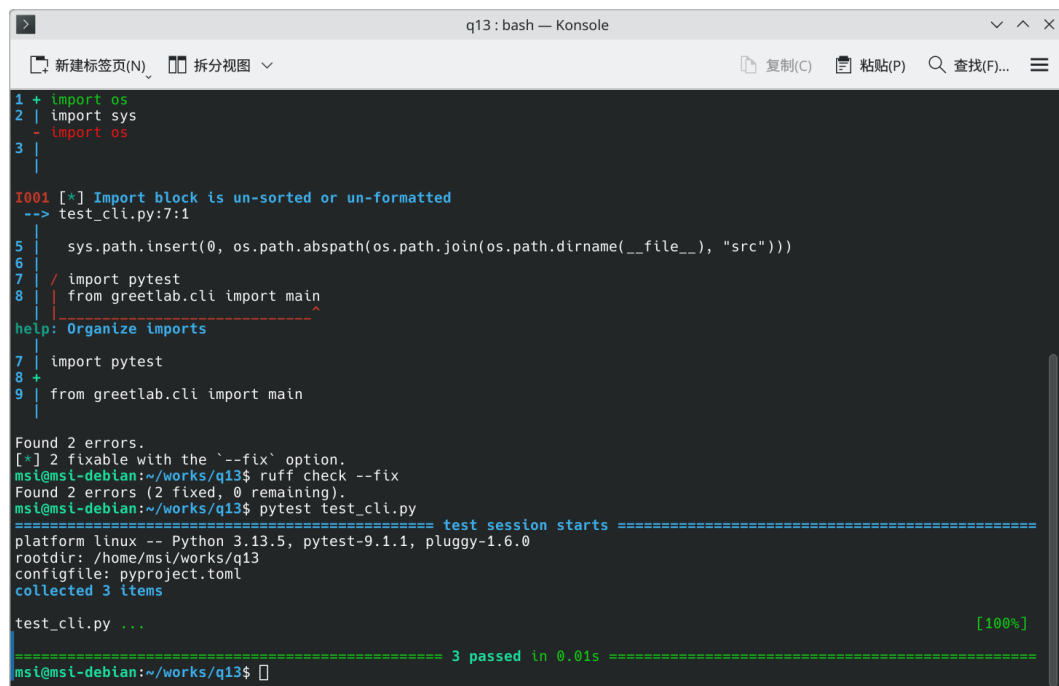
编写 check.sh 并运行

编写 check.sh 依次执行格式检查、静态检查与测试，并保证返回 0：

```
set -e

ruff format
ruff check --fix
pytest
```

运行结果如下：



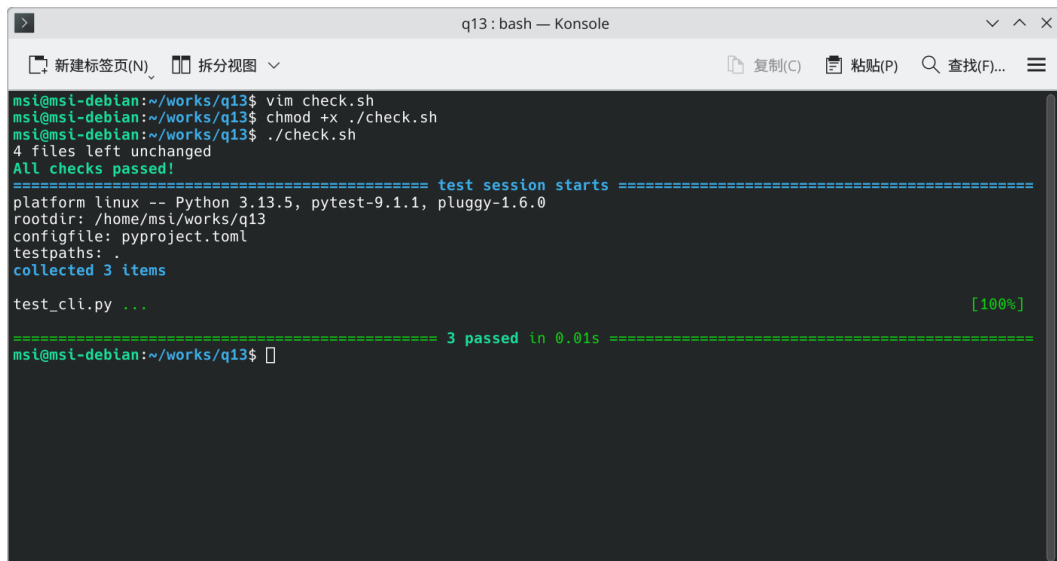
```
q13: bash — Konsole
新建标签页(N)  拆分视图  复制(C)  粘贴(P)  查找(F)...  菜单

1 + import os
2 | import sys
3 | - import os
4 |
I001 [*] Import block is un-sorted or un-formatted
--> test_cli.py:7:1
5 |     sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "src")))
6 |
7 | / import pytest
8 | / from greetlab.cli import main
help: Organize imports
7 | import pytest
8 +
9 | from greetlab.cli import main

Found 2 errors.
[*] 2 fixable with the '--fix' option.
msi@msi-debian:~/works/q13$ ruff check --fix
Found 2 errors (2 fixed, 0 remaining).
msi@msi-debian:~/works/q13$ pytest test_cli.py
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/msi/works/q13
configfile: pyproject.toml
collected 3 items

test_cli.py ... [100%]

===== 3 passed in 0.01s =====
msi@msi-debian:~/works/q13$
```



```
q13: bash — Konsole
新建标签页(N) 拆分视图
msi@msi-debian:~/works/q13$ vim check.sh
msi@msi-debian:~/works/q13$ chmod +x ./check.sh
msi@msi-debian:~/works/q13$ ./check.sh
4 files left unchanged
All checks passed!
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/msi/works/q13
configfile: pyproject.toml
testpaths: .
collected 3 items

test_cli.py ... [100%]

===== 3 passed in 0.01s =====
msi@msi-debian:~/works/q13$
```

1.2 让 Make 只重建真正受影响的产物

本题要求编写 Makefile，只重建真正受影响的产物，具体如下：

- 创建给定的 data.csv、stats.py、build_report.py 和 report.md。
- 编写 Makefile，包含 all、stats.txt、report.txt 和 clean；准确列出数据文件、脚本和中间产物依赖，并把 all、clean 声明为.PHONY。
- 验证首次 make 生成两个产物；第二次无改动时不执行配方。
- touch data.csv 后再次 make，确认重新生成；clean 只删除生成物。

给定内容：

```
# data.csv
name,value
alpha,2
beta,3
```

```
gamma,5

# stats.py
import csv
with open("data.csv") as f:
    total = sum(int(r["value"]) for r in csv.DictReader(f))
open("stats.txt", "w").write(str(total))

# build_report.py
text = open("report.md").read()
total = open("stats.txt").read()
open("report.txt", "w").write(f"{text}\nTotal: {total}\n")

# report.md
# Data Report
```

编写 Makefile

将 all、clean 声明为.PHONY，并为两个产物准确列出依赖：

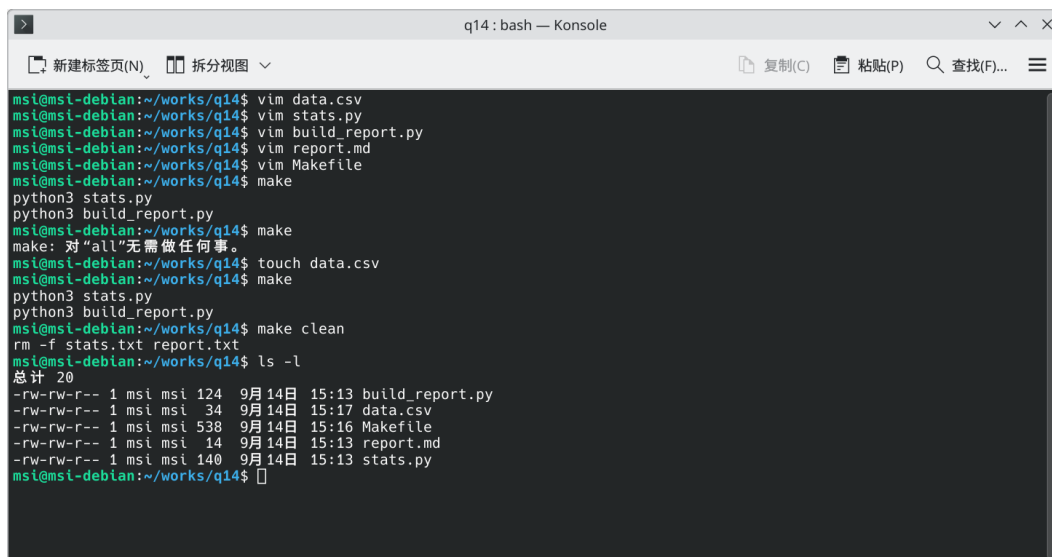
```
# 声明伪目标，防止目录下有同名文件导致 make 误判
.PHONY: all clean

# 默认目标：依赖于两个最终产物
all: stats.txt report.txt

# stats.txt 依赖于源数据 data.csv 和脚本 stats.py
stats.txt: data.csv stats.py
^^Ipython3 stats.py
```

```
# report.txt 依赖于中间产物 stats.txt、脚本 build_report.py 以及模板文件 report.md
report.txt: stats.txt build_report.py report.md
^^Ipython3 build_report.py

# 清理规则：只删除生成的产物，不删除源文件
clean:
^^Irm -f stats.txt report.txt
```



```
q14: bash — Konsole
新建标签页(N)  拆分视图
复制(C)  粘贴(P)  查找(F)...
msi@msi-debian:~/works/q14$ vim data.csv
msi@msi-debian:~/works/q14$ vim stats.py
msi@msi-debian:~/works/q14$ vim build_report.py
msi@msi-debian:~/works/q14$ vim report.md
msi@msi-debian:~/works/q14$ vim Makefile
msi@msi-debian:~/works/q14$ make
python3 stats.py
python3 build_report.py
msi@msi-debian:~/works/q14$ make
make: 对“all”无需做任何事。
msi@msi-debian:~/works/q14$ touch data.csv
msi@msi-debian:~/works/q14$ make
python3 stats.py
python3 build_report.py
msi@msi-debian:~/works/q14$ make clean
rm -f stats.txt report.txt
msi@msi-debian:~/works/q14$ ls -l
总计 20
-rw-rw-r-- 1 msi msi 124  9月14日 15:13 build_report.py
-rw-rw-r-- 1 msi msi  34  9月14日 15:17 data.csv
-rw-rw-r-- 1 msi msi 538  9月14日 15:16 Makefile
-rw-rw-r-- 1 msi msi  14  9月14日 15:13 report.md
-rw-rw-r-- 1 msi msi 140  9月14日 15:13 stats.py
msi@msi-debian:~/works/q14$
```

1.3 把本地 API 数据转换为可读报告

本题要求创建本地 HTTP 服务，再用 curl 与 jq 生成 Markdown 报告，详细如下：

- 把给定 JSON 保存为 packages.json, 并在 q15 目录运行 `python -m http.server 8000`。
- 使用 `curl -fsS` 获取 `http://127.0.0.1:8000/packages.json`。

- 使用 jq 筛选 status 为 active 且 downloads 不少于 100 的记录, 并按 downloads 降序、name 升序排列。
- 编写 api_report.sh, 把结果生成 summary.md, 包含标题和 name/version/downloads 三列表格。

给定 JSON:

```
[
  {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
  {"name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0"},
  {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
  {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
  {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
]
```

启动本地服务并获取数据

启动 HTTP 服务, 再用 curl 获取数据:

```
cd q15
python3 -m http.server 8000 &
curl -fsS http://127.0.0.1:8000/packages.json
```

编写 api_report.sh

使用 jq 筛选并排序, 生成 summary.md:

```
#!/usr/bin/env bash

URL="http://127.0.0.1:8000/packages.json"
OUTPUT="summary.md"
```

```

echo "# API Package Report" > "$OUTPUT"
echo "" >> "$OUTPUT"
echo "| name | version | downloads |" >> "$OUTPUT"
echo "|-----|-----|-----|" >> "$OUTPUT"

curl -fsS "$URL" | jq -r '["." | select(.status == "active" and .downloads >= 100)] | so

```

生成的 summary.md 内容如下:

```
# API Package Report
```

```

| name | version | downloads |
|-----|-----|-----|
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |

```

```

q15: bash — Konsole
新建标签页(N)  拆分视图
msi@msi-debian:~/works/q15$ vim packages.json
msi@msi-debian:~/works/q15$ python3 -m http.server 8000 &
[1] 20583
msi@msi-debian:~/works/q15$ curl -fsS http://127.0.0.1:8000/packages.json | jq '["." | select(.status == "active"
and .downloads >= 100)] | sort_by([-.downloads, .name])[] | " | \(.name) | \(.version) | \(.downloads) |"'
127.0.0.1 - - [14/Sep/2026 15:51:24] "GET /packages.json HTTP/1.1" 200 -
"| delta | 0.9.0 | 450 |"
"| gamma | 1.5.1 | 450 |"
"| alpha | 1.2.0 | 120 |"
msi@msi-debian:~/works/q15$ vim api_report.sh
msi@msi-debian:~/works/q15$ chmod +x ./api_report.sh
msi@msi-debian:~/works/q15$ ./api_report.sh
127.0.0.1 - - [14/Sep/2026 15:57:01] "GET /packages.json HTTP/1.1" 200 -
报告已生成: summary.md
msi@msi-debian:~/works/q15$ cat summary.md
# API Package Report

| name | version | downloads |
|-----|-----|-----|
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
msi@msi-debian:~/works/q15$

```

1.4 修复并交付一个陌生的小型工具仓库

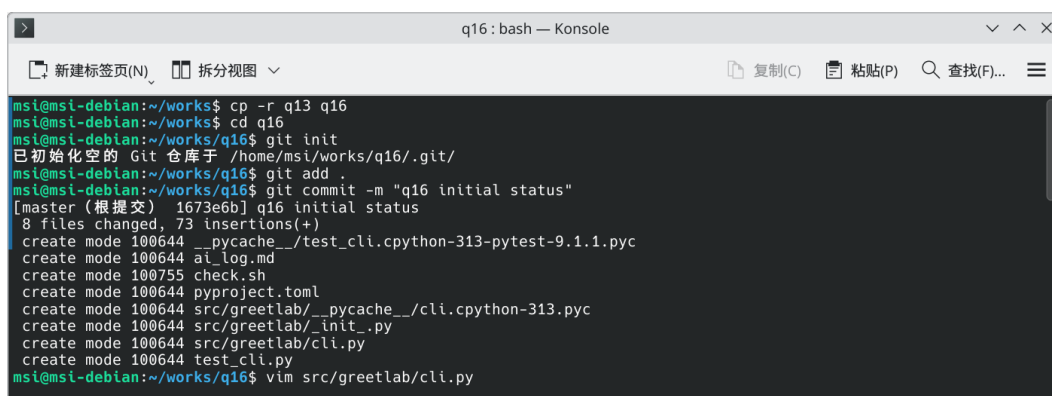
本题要求复用第 13 题项目，完成一次综合检查，具体如下：

- 复制 q13 为 q16 并初始化/继续使用 Git。
- 把问候语实现临时改成始终返回字面量 Hello, name!, 运行 check.sh 确认测试失败；随后定位并修复。
- 编写最小 Makefile, 包含 check、build 和 clean; check 调用 check.sh, build 生成 wheel。
- 运行 make check 和 make build, 计算 wheel 的 SHA-256, 并把最终改动提交为一个内容聚焦的 Git 提交。

初始化仓库并制造失败

复制 q13 为 q16, 初始化 Git, 把实现临时改成始终返回字面量, 运行 check.sh 确认测试失败：

```
cp -r q13 q16 && cd q16
git init
./check.sh    # 临时改坏后应失败
```



```
q16: bash — Konsole
新建标签页(N)  拆分视图
msi@msi-debian:~/works$ cp -r q13 q16
msi@msi-debian:~/works$ cd q16
msi@msi-debian:~/works/q16$ git init
已初始化空的 Git 仓库于 /home/msi/works/q16/.git/
msi@msi-debian:~/works/q16$ git add .
msi@msi-debian:~/works/q16$ git commit -m "q16 initial status"
[master (根提交) 1673e6b] q16 initial status
8 files changed, 73 insertions(+)
create mode 100644 __pycache__/test_cli.cpython-313-pytest-9.1.1.pyc
create mode 100644 ai_log.md
create mode 100755 check.sh
create mode 100644 pyproject.toml
create mode 100644 src/greetlab/__pycache__/cli.cpython-313.pyc
create mode 100644 src/greetlab/_init_.py
create mode 100644 src/greetlab/cli.py
create mode 100644 test_cli.py
msi@msi-debian:~/works/q16$ vim src/greetlab/cli.py
```

定位并恢复 cli.py 中的空白校验逻辑后, make check 通过:

```

q16: bash — Konsole
root@msl-debian:~/works/q16$ ./check.sh
4 files left unchanged
All checks passed!

===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/msl/works/q16
configfile: pyproject.toml
testpaths:
  collected 3 items

test_cli.py F [100%]

===== FAILURES =====
test_normal_name
monkeypatch = <pytest.monkeypatch.MonkeyPatch object at 0x7fee2fcd08>
capsys = <_pytest.capture.CaptureFixture object at 0x7fee38f0eb>

    def test_normal_name(monkeypatch, capsys):
        """测试正常名字，应输出 Hello 并正常退出。"""
        monkeypatch.setattr(sys, "argv", ["-s", "-t", "-greet", "--name", "Alice"])
        main()

        captured = capsys.readouterr()
        assert "Hello, Alice!" in captured.out
        E       AssertionError: assert 'Hello, Alice!' in 'Hello, name!\n'
        E       + where 'Hello, name!\n' = CaptureResult(out='Hello, name!\n', err='').out

test_cli.py:27: AssertionError

===== short test summary info =====
FAILED test_cli.py::test_normal_name - AssertionError: assert 'Hello, Alice!' in 'Hello, name!\n'
===== 1 failed, 2 passed in 0.03s =====
root@msl-debian:~/works/q16$ vim src/greetlab/cli.py
root@msl-debian:~/works/q16$ ./check.sh
4 files left unchanged
All checks passed!

===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/msl/works/q16
configfile: pyproject.toml
testpaths:
  collected 3 items

test_cli.py ... [100%]

===== 3 passed in 0.01s =====
root@msl-debian:~/works/q16$

```

编写 Makefile 并构建

编写最小 Makefile, check 调用 check.sh, build 生成 wheel:

```
.PHONY: check build clean
```

```
check:
```

```
^^I./check.sh
```

```
build:
```

```
^^Ipython3 -m build
```

```
clean:
```

```
^^Irm -rf dist/ build/ *.egg-info .pytest_cache
^^Ifind . -type d -name "__pycache__" -exec rm -rf {} +
```

```
make check
```

```
make build
```

```
q16: bash — Konsole

msi@msi-debian:~/works/q16$ make check
./check.sh
4 files left unchanged
All checks passed!
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/msi/works/q16
configfile: pyproject.toml
testpaths: .
collected 3 items

test_cli.py ... [100%]

===== 3 passed in 0.01s =====

msi@msi-debian:~/works/q16$ make build
python3 -m build
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for sdist...
running egg_info
creating src/greetlab_25020007136.egg-info
writing src/greetlab_25020007136.egg-info/PKG-INFO
writing dependency_links to src/greetlab_25020007136.egg-info/dependency_links.txt
writing entry points to src/greetlab_25020007136.egg-info/entry_points.txt
writing top-level names to src/greetlab_25020007136.egg-info/top_level.txt
writing manifest file 'src/greetlab_25020007136.egg-info/SOURCES.txt'
reading manifest file 'src/greetlab_25020007136.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab_25020007136.egg-info/SOURCES.txt'
* Building sdist...
```

校验和并提交

计算 wheel 的 SHA-256，并把最终改动提交为内容聚焦的 Git 提交：

```
sha256sum dist/greetlab_25020007136-0.1.0-py3-none-any.whl
git add -A
git commit -m "fix: 恢复空白姓名校验并补齐质量门禁"
```

```
q16: bash — Konsole

writing top-level names to src/greetlab_25020007136.egg-info/top_level.txt
reading manifest file 'src/greetlab_25020007136.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab_25020007136.egg-info/SOURCES.txt'
installing to build/bdist.linux-x86_64/wheel
running install
running install_lib
creating build/bdist.linux-x86_64/wheel/greetlab
copying build/lib/greetlab/_init_.py to build/bdist.linux-x86_64/wheel/greetlab
copying build/lib/greetlab/cli.py to build/bdist.linux-x86_64/wheel/greetlab
running install_egg_info
Copying src/greetlab_25020007136.egg-info to build/bdist.linux-x86_64/wheel/greetlab_25020007136-0.1.0-py3.13.egg-info
running install_scripts
creating build/bdist.linux-x86_64/wheel/greetlab_25020007136-0.1.0.dist-info/METADATA
creating /home/msi/works/q16/dist/.tmp-xojsq5r/greetlab_25020007136-0.1.0-py3-none-any.whl and adding 'build/bdist.linux-x86_64/wheel' to it
adding 'greetlab/_init_.py'
adding 'greetlab/cli.py'
adding 'greetlab_25020007136-0.1.0.dist-info/METADATA'
adding 'greetlab_25020007136-0.1.0.dist-info/METADATA'
adding 'greetlab_25020007136-0.1.0.dist-info/entry_points.txt'
adding 'greetlab_25020007136-0.1.0.dist-info/top_level.txt'
adding 'greetlab_25020007136-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
Successfully built greetlab_25020007136-0.1.0.tar.gz and greetlab_25020007136-0.1.0-py3-none-any.whl
msi@msi-debian:~/works/q16$ sha256sum dist/*.whl
9a132f6d25ced9c9963463f9b2a80fca115178ad84c5feae349b63ba168 dist/greetlab_25020007136-0.1.0-py3-none-any.whl

msi@msi-debian:~/works/q16$

q16: bash — Konsole

msi@msi-debian:~/works/q16$ git status
位于分支 master
未跟踪的文件:
  (使用 "git add <文件>..." 更新要提交的内容)
  (使用 "git restore <文件>..." 丢弃工作区的改动)
新文件:
  src/greetlab/_pycache_/_cli.cpython-313.pyc
  dist/
  src/greetlab_25020007136.egg-info/
修改后准备提交:
  (使用 "git add <文件>..." 以包含要提交的内容)
  dist/
  src/greetlab_25020007136.egg-info/
修改后未加入提交 (使用 "git add" 和/或 "git commit -a")
msi@msi-debian:~/works/q16$ git add src/greetlab/_cli.py Makefile
msi@msi-debian:~/works/q16$ git commit -m "Fixed the bug in greeting & add Makefile"
[master 9521a13] Fixed the bug in greeting & add Makefile
1 file changed, 14 insertions(+)
create mode 100644 Makefile
msi@msi-debian:~/works/q16$ git log --oneline -2
052a11 (HEAD -> master) Fixed the bug in greeting & add Makefile
1674bda q16 initial status
msi@msi-debian:~/works/q16$
```

2 课后练习

2.1 代码质量

2.1.1 在 pyproject.toml 中配置 ruff 规则

ruff 的规则与格式化参数可在 pyproject.toml 中集中配置，避免在命令行反复指定。

```
[tool.ruff]
line-length = 88
target-version = "py313"
[tool.ruff.lint]
select = ["E", "F", "I", "W"]
[tool.pytest.ini_options]
testpaths = ["."]
python_files = "test_*.py"
```

2.1.2 用 pytest 参数化测试减少重复

使用 pytest.mark.parametrize 可以对同一测试函数批量提供多组输入，避免复制粘贴。

```
import pytest

@pytest.mark.parametrize("name", ["Alice", "Bob", "Carol"])
def test_normal_names(name, capsys, monkeypatch):
    monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", name])
    main()
    assert f"Hello, {name}!" in capsys.readouterr().out
```

2.1.3 用 `pytest.raises` 断言异常与退出码

对会抛出异常的代码，用 `pytest.raises` 既能捕获异常，又能断言退出码，是测试 CLI 参数校验的常用方式。

```
import pytest

def test_blank_name(monkeypatch):
    monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", ""])
    with pytest.raises(SystemExit) as exc_info:
        main()
    assert exc_info.value.code == 2
```

2.2 元编程：构建系统

2.2.1 GNU Make 的自动变量

自动变量 `$@`、`$<`、`$^` 分别代表目标文件、第一个依赖和全部依赖，可让配方更通用。

```
output.txt: input.txt
^^Icat $< > $@      # $< 即 input.txt, $@ 即 output.txt
```

2.2.2 用 `.PHONY` 声明伪目标

当目标名与文件名同名时，`make` 可能误判其已是最新；用 `.PHONY` 声明后，每次都会执行该配方。

```
.PHONY: all clean build
all: build
clean:
^^Irm -rf dist/ *.egg-info
```

2.3 大杂烩：API、jq 与 Markdown

2.3.1 curl 的常用选项

-f 在 HTTP 错误时静默失败，-s 关闭进度条，-o 保存到文件，三者组合适合脚本化的 API 调用。

```
curl -fsS http://127.0.0.1:8000/packages.json
curl -fsS http://127.0.0.1:8000/packages.json -o packages.json
```

2.3.2 用 jq 筛选与排序 JSON

jq 的 select 用于按条件过滤，sort_by 可按一个或多个键排序，负号表示降序。

```
jq '[] | select(.status == "active" and .downloads >= 100) | sort_by([-.downloads, .name])'
```

2.3.3 用 jq -r 拼接 Markdown 表格

-j 使用原始输出，配合字符串拼接可直接生成 Markdown 表格行。

```
jq -r '[] | "| \(.name) | \(.version) | \(.downloads) |"'
```

2.4 Python 与 CLI 约定

2.4.1 用 argparse 定义必选参数

argparse 的 required=True 可声明必选参数，-help 由框架自动生成，符合常见 CLI 约定。

```
import argparse
```

```
p = argparse.ArgumentParser()
p.add_argument("--name", required=True)
a = p.parse_args()
print(f"Hello, {a.name}!")
```

2.4.2 计算文件的 SHA-256 校验和

使用 sha256sum 或 Python 的 hashlib 计算文件摘要，可用于校验构建产物的完整性。

```
sha256sum dist/greetlab_25020007136-0.1.0-py3-none-any.whl
```

```
import hashlib
h = hashlib.sha256(open("file.whl", "rb").read())
print(h.hexdigest())
```

3 感悟

通过第四周的实验练习，形成以下认识和体会：

- 质量门禁的价值：把 ruff 与 pytest 的配置固化到 pyproject.toml，再用一个 check.sh 串起格式、静态检查与测试，能让“是否可交付”变得自动化、可重复，而不再依赖个人记忆。
- 构建系统的依赖观：Makefile 的核心是准确描述产物与来源之间的依赖关系，从而只重建真正受影响的部分；.PHONY 则用于区分伪目标与真实文件名，避免误判。
- 命令行工具的组合能力：curl 负责取数、jq 负责筛选与排序、Markdown 负责呈现，三者通过管道串联即可把一份 JSON 变成可读报告，体现了 Unix 工具哲学。

- 综合交付的完整链路：从临时制造失败、定位修复，到 make check、make build、计算 SHA-256 校验和，再到内容聚焦的 Git 提交，串起了一个陌生仓库从“不确定状态”到“可交付”的全过程。

代码质量、元编程与命令行数据处理共同点，在于用约定和自动化把重复性、易出错的人工步骤转化为可验证的流程。这也是“不止于代码”的意义所在：衡量工程能力的，不仅是能否写出正确实现，更是能否建立一套让正确性可复现、可校验、可交付的机制。

Github 提交

此处上传 Github 提交记录。(报告于完成后上传)

